

CardSmith

CardSmith (*student proposal*)

Assessment: ✓ Fit · **Category:** Games · **Assessed:** 2026-09-21 · **Status:** awaiting instructor approval

Proposal (as submitted): *CardSmith* is a fantasy-themed, single-player PvE **card battler**: players manipulate elemental cards to defeat enemies through resource management and combo-driven combat. Players manage **Energy** by choosing to play cards for their effects or **discard them to generate resources**, and combine cards from four elements (Fire, Water, Earth, Lightning) to discover **Elemental Combinations** that modify attacks. The game has linear level progression, adaptable enemy behaviours, deck customisation, a permanent card collection and upgrade systems. Designed for **native mobile**. Team: 6 CS + 1 Design + 2 Art. Named technical features: a debugging UI; adaptable enemy AI; haptic feedback; a combo-driven combat system. Load-bearing C++: a **deterministic, turn-based C++ simulation loop** running each fixed step, giving real-time feedback for animations and VFX. Tools: a **combo-system editor** (define/test/simulate combos), a **toggleable in-game debug overlay** (state inspector, combo logs), a **haptic-pattern previewer** (assign a haptic pattern to an event), and an **enemy encounter editor** (define enemy type, HP, AI behaviour, modifiers per encounter). Design features: elemental combo-driven combat; discard-to-Energy resource management; progressive PvE encounters with waves + unique enemy behaviours; and deck customisation + permanent collection + a Tech Tree.

References:

- [Slay the Spire on Steam](#) (the benchmark single-player PvE deckbuilder: build a deck, chain card combos, fight structured encounters, and unlock permanent progression — the deckbuilder-PvE + permanent-collection precedent)

- Monster Train on Steam (a strategic deckbuilder whose lifeblood is **synergy/ combos** between cards and factions — the combo-discovery + deck-synergy precedent)
- Magicka on Steam (a game built on **combining elements** — Fire, Water, Earth, Lightning and more — into new effects; the elemental-combination system precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✕ Not needed | » Second trimester

Core Tech Requirement	CardSmith
External Database	✓
Authentication »	✓
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✕
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — the stated target, with haptic feedback; device: Pixel 9a) — the Native PC build is the development/tools vehicle (the combo editor, encounter editor, debug overlay and haptic previewer). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts and the permanent collection / Tech-Tree persistence.

Front end: A 2D renderer for cards, the hand/board, enemies, the Energy/combo HUD and the elemental-combination VFX, plus card animations and the deck/collection/Tech-Tree screens; haptics paired to key events.

Languages: Native — Kotlin Native Android UI + the C++ engine via JNI

(Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The deterministic, turn-based C++ simulation loop (card play/discard, Energy, the combo resolver, the adaptable enemy AI, damage), driving animations/VFX/haptics; plus Firebase for accounts + permanent collection/Tech-Tree persistence. **Languages:** C++ (engine + sim); Firebase-backed services on the Extended tier.

Backend scope: Local now, server-ready (T2) — the accounts / collection / Tech-Tree seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ loop is load-bearing — each fixed step it resolves card plays/discards, the Energy economy, the elemental-combination combos and the adaptable enemy AI deterministically, then feeds animations, VFX and haptics.
- *Data-driven:* cards, the four elements + their combination rules, enemy types/behaviours, encounters (type/HP/AI/modifiers) and the Tech-Tree are all data — a new card, combo or encounter is a data change (authored in the combo + encounter editors).
- *2D:* card/board/enemy rendering, card animations, the Energy/combo HUD, the elemental-combination VFX and the deck/collection/Tech-Tree screens (custom-rendered). *3D:* not needed. *Networking:* single-player; the collection/Tech-Tree persistence is async client↔cloud (Application Communication ✓), so a Networking System is optional (●).
- *Custom tool:* the **combo-system editor** (define/test/simulate combos) and the **enemy encounter editor** (type/HP/AI/modifiers) are content-authoring tools; the **debug overlay** (state inspector + combo logs) and the **haptic-pattern previewer** are strong dev tools.
- *External DB:* accounts + the permanent card collection + Tech-Tree progression. *Auth »:* accounts for persistent progression. *Analytics »:* optional — combo usage / encounter win-rates for balancing.

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the deterministic turn-based combat sim + card input + haptics.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a deterministic fixed-step turn-based loop resolving card plays/discards, Energy and combo resolution, with touch card input and event-triggered haptics. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI, device haptics); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** None — foundation of this project.
- **Feeds into:** Every other goal; the enemy AI (M1G8) and combo VFX (M1G13) run on its tick.

- **G2 ★ Data-Driven Architecture of Core Objects / Entity System**

Tech & dependencies

- **Tech:** Entity/component model with cards, the four elements + combination rules, enemy types/behaviours, encounters and the Tech-Tree in hot-reloadable JSON — a new card, combo or encounter is a data change.
- **Prerequisites:** G1 (loop & update hooks).
- **Feeds into:** M2G2 (the combo editor), M1G8 (enemy data), M2G13 (combo/deck systems).

- **G3 ★ Debugging, Profiling, Stress Test & CI** — home of the toggleable debug overlay (state inspector + combo logs).

Tech & dependencies

- **Tech:** ImGui debug overlay (state inspector, combo logs, deterministic-replay), Tracy profiling of a busy combat turn with combo VFX, Catch2 tests for the deterministic combo/turn resolution and data loading, on a GitHub Actions Windows+Android matrix.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G3 (CI hardening); the in-game debug overlay is a named deliverable.
- **G4 2D Graphics** — the card/board/enemy renderer, card animation and elemental-combination VFX + HUD.
Tech & dependencies
 - **Tech: Web build:** Sprite renderer for cards, hand/board, enemies, the Energy/combo HUD and card animations; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** G1 (loop & window).
 - **Feeds into:** M2G4 (advanced combo VFX); every visible frame.
- **G8 Enemy AI & Pathfinding** — the **adaptable enemy AI** (responds to the player's cards/deck).
Tech & dependencies
 - **Tech:** Finite-state / utility AI that reads the board and the player's likely plays and adapts intents (block, buff, target the biggest threat), tuned per enemy type from data (custom C++).
 - **Prerequisites:** G1 (tick, turn state), G2 (enemy data).
 - **Feeds into:** M2G10 (advanced adaptive AI); the progressive-encounter challenge.
- **G13 Particle & VFX System** — the elemental-combination VFX (Fire/Water/Earth/Lightning combos), object-pooled.
Tech & dependencies
 - **Tech:** A pooled particle/VFX system (emitters, color/scale over lifetime) that renders each elemental-combination's signature effect, driven from data so new combos get VFX without code.
 - **Prerequisites:** G1 (loop), G4 (rendering).
 - **Feeds into:** M2G4 (advanced VFX); the combo payoff feedback (with haptics).

M2:

- **G1 ★ Architecture — Optimization & Platform Validation**

Tech & dependencies

- **Tech: Web build:** Tracy profiling of a combo-heavy turn, ASan on MSVC, pools for cards/particles, and on-device validation (Pixel 9a, incl. haptics); the Firebase/Linux-server seam (accounts + collection + Tech-Tree) set up (account + software + basic connectivity test) by M2.
- **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + persistent progression.
- **G2 ★ Content Editor — Core (Combo-System Editor)** — define, test and simulate elemental combinations.

Tech & dependencies

- **Tech:** Dear ImGui editor to author element-combination rules and their effects/VFX, run a simulated resolution against a test hand/enemy, and inspect the result, with round-trip JSON and undo/redo. (The enemy encounter editor — type/HP/AI/modifiers — is built on the same tool tech.)
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (card/combo data), M1G13 (VFX for preview).
- **Feeds into:** All combo/encounter content; the custom-tools showcase.
- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android Debug builds, tests (deterministic combo/turn resolution, AI, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.
- **G8 Firebase — Accounts, Leaderboards & Analytics** — accounts + the permanent card collection + Tech-Tree persistence.

Tech & dependencies

- **Tech:** Firebase Auth accounts, Firestore for the permanent card collection, deck configurations and Tech-Tree unlocks synced across devices, and optional Analytics, via the REST API from C++.
 - **Prerequisites:** G2 (collection/Tech-Tree data model).
 - **Feeds into:** Permanent progression (design feature 4); cross-device continuity.
- **G10 AI — Advanced** — the adaptable enemy AI matured across the progressive encounters.

Tech & dependencies

- **Tech:** Behaviour trees / deeper utility AI that adapts to the player's deck and current board, plus difficulty tiers and distinct enemy roles across waves; tuned from data.
 - **Prerequisites:** M1G8 (adaptable-AI foundation).
 - **Feeds into:** The increasing challenge that rewards adaptation (design feature 3).
- **G13 Gameplay Systems II** — the **combo / discard-to-Energy / deck / collection / Tech-Tree** systems matured.

Tech & dependencies

- **Tech:** C++ systems over the ECS for the elemental-combination resolver, the discard-to-Energy economy, deck customisation, the permanent collection and the Tech-Tree upgrades; balancing-data-driven.
- **Prerequisites:** M1G2 (card/combo data), M1G1 (combo resolution).
- **Feeds into:** The strategic depth + progression at showcase; T2 card/combo expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 1 Design + 2 Art (9) — matches the team's stated make-up. One designer owns the elemental-combination system, the discard-to-Energy economy, the deck/collection/Tech-Tree progression and the adaptable-AI/encounter difficulty design; two artists carry the card art (a fantasy elemental set), the enemy roster, the elemental-combination VFX and the deck/collection/Tech-Tree UI — a card game is art-forward (every card is a bespoke illustration), which justifies the two artists (rules: min 1 Design mandated, max 9 total, $D+A \leq 3$; see [team-formation.pdf](#)). *Scope note (advisory):* a deterministic combo engine + adaptable AI + deck/collection/Tech-Tree + a combo editor + an encounter editor is a full but well-structured scope for a single designer — the elemental-combo combat with the discard-to-Energy loop, ~2 elements' worth of combos, a few encounters and the combo editor is the M1/M2 floor; all four elements, the full Tech-Tree and broad card variety are the stretch. With one designer, keep the card/combo/encounter **balance** tightly scoped.

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents.

- **Design M1:**

- M1D1 ★ Features Design Document (the elemental-combo card battler, the discard-to-Energy loop and the collection/Tech-Tree progression; primary benchmark e.g. Slay the Spire / Monster Train, with a Singapore benchmark for the local-awareness criterion)
- M1D2 Core Gameplay Loop Prototype (a greybox proving play-vs-discard + elemental combos makes a satisfying turn) · prototype in Unity
- M1D6 Systems & Economy Design (the four elements + combination rules, the discard-to-Energy economy, and the deck/collection/Tech-Tree economy — every system with a purpose)

- **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D3 Systems Balance & Tuning — Data-Driven (card/combo/enemy and Tech-Tree balance via the team's JSON, no code changes)

- **UI/UX M1** (graded by the Design instructor for the Games category):
 - M1U1 ★ UI Standards & Wireframe Baseline — readability standards for the hand/board, the Energy/combo indicators and the deck/collection/Tech-Tree screens.
- **UI/UX M2:**
 - M2U1 ★ Functional UI & Usability — battle, deck-builder, collection and Tech-Tree screens navigable unaided on the Native-Android target.
 - M2U2 ★ UX Validation & Final Polished UI — validated, polished combat + combo-discovery UX (with haptics) tested with players on the Pixel 9a.
- **Art M1:** 2 artists (card illustrations + enemies + elemental VFX + UI)
 - M1A1 ★ Art Bible / Proof of Concept (the fantasy elemental visual direction — palette per element, card-frame language)
 - M1A2 Character Concepts & Model Sheets (the enemy roster + any bosses)
 - M1A3 Environment, Props & VFX Concepts (battle backdrops + the elemental-combination VFX concepts)
 - M1A4 UI Art Concepts & Typography (card art/frames, Energy/ combo HUD, deck/collection/Tech-Tree screens)
 - M1A5 Animation Foundations (card play/draw/discard, combo triggers, enemy attacks)
 - M1A6 Early Asset Production — Prototype Support
- **Art M2:**
 - M2A1 ★ Critical Asset Production & In-Engine Visual Quality
 - M2A2 Character Animation — Extensive (enemy attacks/reactions, boss beats)
 - M2A3 Environment Alive — Ambient Animation & Layering (living battle backdrops)
 - M2A4 UI Art — States & Feedback (card-hover/played/discarded, Energy, combo-ready, Tech-Tree states)
 - M2A5 VFX — Critical Effects & Transitions (the elemental-combination effects — the core visual payoff — plus hit/defeat VFX)
 - M2A6 Cutscenes & Storyboards (fantasy level/story beats)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (fantasy BGM, card-play/draw/discard, per-element and combo cues), suitable and correctly triggered, and paired with haptics on key events.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for overlapping combo SFX.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (fantasy BGM + card / per-element / combo / enemy / defeat SFX, ≥6 types), the combo payoff cues coordinated with the haptic feedback.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; a distinct cue per elemental combination).
- M2TP1 ★ Final Presentation — 7 min in front of class + instructors: live custom-tools showcase (the combo editor + encounter editor), prototype demo, team post-mortem (wk14)

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-21.*